

Phishing Incident Response — Part 2:

Phishing Incident Response — Option 2: User Clicked the Link

I responded to a simulated phishing-click incident, building a Suricata-to-Wazuh detection pipeline from scratch to confirm the alert reached the dashboard, then executed the full NIST incident response lifecycle to contain and restore the affected host.

1. Overview

This engagement completes the second half of the phishing simulation scenario introduced in Portfolio Part 7. Where Part 7 covered the analyst-side triage of a reported suspicious email (Option 1), this part simulates the more serious outcome: a user who clicked the phishing link rather than reporting it. The scenario demonstrates what happens after a click — from initial detection through containment, eradication, and recovery — following the NIST Incident Response lifecycle as a guiding framework.

A key additional achievement in this scenario was the integration of Suricata's eve.json alert feed into the Wazuh SIEM pipeline. At the start of this session, Suricata was detecting attacks but Wazuh was completely unaware of them. By the end, a Suricata alert generated by the phishing click was visible in the Wazuh dashboard as rule 86601 within seconds of the connection being made. The steps taken to build that integration, and the important concepts behind why it required multiple components to work, are documented fully in Section 5.

This scenario also surfaced a recurring and important lab insight: running both VMs simultaneously under memory pressure causes kernel-level soft lockup crashes. A revised startup procedure (ubuntu-victim fully loaded before Kali is booted, DVWA and modsec-waf stopped when not needed for the scenario) significantly reduced resource contention and is now the documented operating procedure for this lab.

2. NIST Incident Response Lifecycle — Phase Coverage

NIST Phase	Coverage in This Scenario
Preparation	Lab startup procedure refined to minimize RAM contention; Suricata systemd timeout fixed (TimeoutStartSec=900s); Suricata-to-Wazuh integration built from scratch; monitoring terminals set up before attack window opened
Detection & Analysis	Suricata detected the phishing connection via ET Open signature 2034636; Wazuh received and alerted via rule 86601; full alert metadata captured in dashboard
Containment	UFW deny rule applied to block ubuntu-victim from making any further outbound connections to attacker IP 10.0.2.15
Eradication & Recovery	Containment rule removed; firewall disabled; system confirmed restored to clean baseline
Post-Incident Activity	Suricata-to-Wazuh integration permanently added to the lab; portfolio documentation produced; lab coverage gaps identified for future remediation

3. Lab Architecture

This scenario required both VMs running simultaneously, but only for the shortest possible window. All non-essential services (DVWA, modsec-waf) were stopped before Kali was booted to maximise available RAM.

Node	Role	Services Active During Attack Window
ManKali-attacker (Kali)	Attacker VM	Python HTTP server on port 8888 hosting fake Microsoft login page
ubuntu-victim (Ubuntu 24.04)	Victim VM	Suricata IDS (ET Open, 52,710 rules), Wazuh agent (logcollector reading eve.json)

Node	Role	Services Active During Attack Window
wazuh-manager (DigitalOcean)	SIEM	Wazuh manager/indexer/dashboard receiving and alerting on Suricata events forwarded by the agent

4. Scenario Setup

4.1 Lab startup procedure (refined this session)

Following two kernel soft-lockup crashes in prior sessions caused by resource exhaustion when both VMs started together, the startup procedure was revised:

- Boot ubuntu-victim first with Kali powered off
- Apply the Suricata systemd timeout fix: TimeoutStartSec=900s via `/etc/systemd/system/suricata.service.d/timeout.conf`
- Start services manually in order: docker → (dvwa if needed) → suricata
- Wait for Suricata to fully load the ET Open ruleset (~90 seconds without Kali competing for resources)
- Stop DVWA and modsec-waf if not needed for the scenario (freed ~400MB RAM)
- Only then boot Kali — minimizing the window both VMs run simultaneously

4.2 Phishing infrastructure (Kali)

The phishing page from Part 7 was already in place on Kali. The Python HTTP server was started immediately on boot:

```
cd ~/phishing-site && python3 -m http.server 8888
```

This served a fake Microsoft Account sign-in page at `http://10.0.2.15:8888/index.html`, replicating the credential-harvesting landing page the phishing email from Part 7 pointed to.

4.3 Monitoring setup (ubuntu-victim)

Two monitoring streams were active before the attack:

- Suricata live alert stream: `sudo tail -f /var/log/suricata/eve.json | grep --line-buffered '"event_type":"alert"'`
- Wazuh dashboard open at `https://204.48.20.38`, Threat Hunting view, filtered to ubuntu-victim, last 10 minutes

5. Building the Suricata-to-Wazuh Integration

This section documents a significant piece of work completed during this scenario: wiring Suricata's alert output into the Wazuh SIEM pipeline so that Suricata detections appear as Wazuh alerts in the dashboard. At the start of this session, Suricata was detecting attacks silently with no Wazuh visibility. By the end, the full pipeline was confirmed working end-to-end.

5.1 Why Wazuh was not alerting on Suricata events

Wazuh's log processing pipeline has three distinct stages, and all three must be in place before an alert appears in the dashboard:

- Log ingestion — the logcollector must be configured to read the file
- Decoding — Wazuh must know how to parse the file format and extract structured fields from each event
- Rule matching — a rule must exist that fires on the decoded event data

The eve.json localfile block had been added to ossec.conf, and the logcollector confirmed it was reading the file ('Analyzing file: /var/log/suricata/eve.json'). But without a decoder, Wazuh was ingesting raw text it couldn't interpret, and without a matching rule, even decoded events would be silently dropped. All three stages were missing or incomplete.

This is an important concept for a SOC analyst: 'the tool is reading the log' does not mean 'the tool is alerting on the log.' Log ingestion, decoding, and rule matching are independent steps in Wazuh's pipeline, and each must be verified separately.

5.2 Step 1 — Confirm logcollector was reading eve.json

```
sudo tail -50 /var/ossec/logs/ossec.log | grep -i "eve|suricata|logcollector"
```

Output confirmed: '(1950): Analyzing file: /var/log/suricata/eve.json' — ingestion was working. The problem was downstream.

5.3 Step 2 — Add a Suricata decoder

A custom decoder was created to tell Wazuh how to parse Suricata's JSON format:

```
sudo mkdir -p /var/ossec/etc/decoders
sudo tee /var/ossec/etc/decoders/suricata_decoder.xml << 'EOF'
<decoder name="Suricata-Eve-JSON">
  <prematch>{"timestamp"</prematch>
</decoder>

<decoder name="Suricata-Eve-JSON-event">
  <parent>Suricata-Eve-JSON</parent>
  <plugin_decoder>JSON_Decoder</plugin_decoder>
</decoder>
EOF
```

The prematch pattern identifies Suricata eve.json lines by their opening '{"timestamp"' string. The JSON_Decoder plugin then extracts all fields (event_type, alert.signature, src_ip, dest_ip, etc.) into searchable Wazuh fields.

5.4 Step 3 — Add a Suricata rule

A custom rule was created to generate an alert whenever a decoded Suricata event had event_type: alert:

```
sudo mkdir -p /var/ossec/etc/rules
sudo tee /var/ossec/etc/rules/suricata_rules.xml << 'EOF'
<group name="suricata,">
  <rule id="86600" level="7">
    <decoded_as>Suricata-Eve-JSON</decoded_as>
    <field name="event_type">alert</field>
    <description>Suricata alert: $(alert.signature)</description>
    <group>ids,suricata,</group>
  </rule>
</group>
EOF
```

5.5 Why the alert appeared as rule 86601 (level 3) rather than 86600 (level 7)

After restarting the Wazuh agent, the phishing click generated a Wazuh alert — but it appeared as rule 86601 at level 3, not the custom rule 86600 at level 7. This is because Wazuh already had a built-in rule 86601 for Suricata events that matched first and took priority over the custom rule.

The level 3 severity (informational) reflects how the built-in rule classifies generic Suricata INFO-category signatures. The custom rule 86600 would have set level 7, but Wazuh's rule processing matched the built-in rule first. This is expected Wazuh behaviour — built-in rules take precedence unless explicitly overridden.

The important finding is that the full pipeline now works. The level classification is a tuning consideration, not a fundamental gap. Custom rules can be adjusted to override the built-in severity if needed for specific signature categories.

6. The Simulated Attack — User Clicks the Phishing Link

6.1 Attack execution

From ubuntu-victim, the user's click was simulated using curl, generating real cross-VM HTTP traffic that Suricata could inspect:

```
curl -v http://10.0.2.15:8888/index.html
```

The connection was established successfully: HTTP/1.0 200 OK, full phishing page HTML returned. This represents the victim's browser loading the credential-harvesting page.

6.2 Suricata detection — live alert

Suricata fired immediately on the connection. The live alert stream on ubuntu-victim printed the following event:

Field	Value
Timestamp	2026-09-28T18:17:12 UTC
Signature ID	2034636
Signature	ET INFO Python SimpleHTTP ServerBanner
Category	Misc activity
Source IP	10.0.2.15 (Kali attacker)
Source Port	8888
Destination IP	10.0.2.3 (ubuntu-victim)
Protocol	TCP / HTTP
Action	Allowed (IDS mode — detect only, not block)
Confidence	High

The ET Open ruleset detected the Python SimpleHTTP server banner in the HTTP response from Kali — a realistic detection of an attacker using a common lightweight HTTP server to host phishing infrastructure.

6.3 Wazuh dashboard alert

For the first time in this lab, a Suricata alert was visible in the Wazuh dashboard. Rule 86601 fired at 13:17:20 local time (18:17 UTC) with the following key fields confirmed in the Document Details panel:

Wazuh Field	Value
agent.name	ubuntu-victim
agent.ip	10.0.2.3
data.alert.signature	ET INFO Python SimpleHTTP ServerBanner
data.alert.signature_id	2034636
data.alert.category	Misc activity
data.alert.confidence	High
data.alert.action	allowed
data.app_proto	http
rule.id	86601
rule.level	3

7. NIST Phase 3 — Containment

Following confirmation of the phishing connection, containment was applied by enabling the host-based firewall and blocking any further outbound connections from ubuntu-victim to the attacker's IP:

```
sudo ufw enable
sudo ufw deny out to 10.0.2.15
sudo ufw status
```

UFW status confirmed: Firewall active, rule applied — 10.0.2.15 DENY OUT Anywhere. This simulates the standard SOC containment action of isolating a compromised host from attacker infrastructure while investigation continues. Any further attempt by the victim to connect to the phishing server or attacker's IP would be dropped at the firewall level.

8. NIST Phase 4 — Eradication & Recovery

Since this was a simulated click with no actual payload delivered (the phishing page served HTML, not an executable or script), the eradication step was straightforward. In a real incident this phase would include checking for dropped files, persistence mechanisms, credential exposure, and lateral movement indicators.

The containment rule was removed and the firewall disabled to restore normal lab operation:

```
sudo ufw delete deny out to 10.0.2.15
sudo ufw disable
```

Output confirmed: Rule deleted. Firewall stopped and disabled on system startup. System restored to known clean baseline.

9. Detection Pipeline — End-to-End Confirmation

This scenario confirmed the full detection chain for the first time in this lab:

Stage	Component	Evidence
Attack	Kali Python HTTP server (port 8888)	curl from ubuntu-victim returned HTTP 200 with phishing page HTML
Network detection	Suricata 8.0.3 with ET Open ruleset	Signature 2034636 fired live on eve.json alert stream
Log ingestion	Wazuh logcollector reading /var/log/suricata/eve.json	Confirmed via ossec.log: 'Analyzing file: /var/log/suricata/eve.json'
Decoding	Custom Suricata-Eve-JSON decoder	JSON fields extracted and matched by built-in rule 86601
SIEM alert	Wazuh dashboard rule 86601	Alert visible at 13:17:20 local with full alert metadata

10. Key Findings

- The Suricata-to-Wazuh pipeline requires three independent components: log ingestion (localfile block in ossec.conf), a decoder (telling Wazuh how to parse eve.json JSON format), and a matching rule (telling Wazuh what to do with decoded events). All three must be present before an alert appears in the dashboard.
- Wazuh already had a built-in Suricata rule (86601, level 3) that matched before the custom rule (86600, level 7) could fire. This is expected Wazuh rule precedence behaviour, not a failure. The integration is working correctly; severity tuning is a separate configuration task.
- ET Open signature 2034636 detected the Python SimpleHTTP server banner in the phishing server's HTTP response — a real, signature-based detection of attacker infrastructure, not a protocol anomaly or heuristic.
- Memory management on the 8GB host is critical. Stopping DVWA and modsec-waf before booting Kali freed enough RAM to prevent the kernel soft-lockup crashes that had caused forced power-offs in prior sessions. The revised startup procedure is now the documented operating norm for this lab.
- The systemd TimeoutStartSec fix (900s) successfully prevented the Suricata startup timeout that had failed the first boot attempt in the previous session.

11. MITRE ATT&CK Mapping

Tactic	Technique	Detail
Initial Access	T1566.001 — Spearphishing Link	User simulated clicking the embedded phishing URL from the Part 7 email artifact
Command and Control	T1071.001 — Web Protocols	Phishing page served over HTTP from attacker-controlled Python server on port 8888
Defense Evasion	T1036 — Masquerading	Phishing page presented as a Microsoft Account sign-in portal to deceive the user
Credential Access	T1056 — Input Capture (attempted)	Credential-harvesting form embedded in phishing page — no credentials submitted in this simulation

12. Lab Coverage Gaps

As committed at the start of the phishing scenario, the following honest gaps are documented now that both Option 1 and Option 2 are complete:

- No real email delivery — the phishing email was a crafted .eml artifact, not a live email delivered via SMTP. A real lab would include a mail server (e.g. Postfix) and email client to test gateway-level filtering.
- No Windows victim — real phishing campaigns target Windows and Outlook. This lab's Linux-only victim cannot simulate macro execution, Windows credential theft, LSASS dumping, or endpoint telemetry via Windows Event Logs and Sysmon. A Tier 3 Windows VM addition is planned.
- No EDR layer — Wazuh provides SIEM and basic FIM, but there is no endpoint detection and response (EDR) tool such as CrowdStrike or Microsoft Defender for Endpoint. An EDR would provide process-level telemetry that Wazuh alone does not.
- No email gateway alerts — in a real organisation, a phishing email would generate alerts from an email security gateway (Proofpoint, Mimecast, Microsoft Defender for Office 365) before the user ever saw it. That layer does not exist in this lab.
- Suricata in IDS mode only — Suricata is running in detection mode (IDS), not inline prevention mode (IPS). It alerts on the phishing connection but does not block it. Configuring Suricata as an IPS would be a meaningful defensive addition.

13. Skills Demonstrated

- Full NIST IR lifecycle execution across a realistic phishing scenario: preparation, detection and analysis, containment, eradication and recovery, and post-incident documentation
- Building a Suricata-to-Wazuh detection pipeline from scratch: diagnosing why ingestion alone is insufficient, creating a custom JSON decoder, adding a rule, restarting the agent, and verifying the alert end-to-end in the dashboard
- Understanding and explaining Wazuh's three-stage log processing model (ingestion, decoding, rule matching) and why each stage must be verified independently
- Real-time network intrusion detection using Suricata ET Open signatures, confirmed via live eve.json alert stream and Wazuh dashboard correlation
- Host-based containment using UFW firewall rules to isolate a compromised host from attacker infrastructure
- Lab resource management: diagnosing kernel soft-lockup root cause (RAM exhaustion from concurrent VM load), implementing a revised startup procedure, and applying a systemd timeout fix to prevent recurrence